

Crash-Consistent Checkpointing for Exactly-Once Agent Effects: A Write-Ahead Claim Log for LLM Agent Harnesses

Mohammed Kamran Syed

Abstract—LLM agent harnesses checkpoint *trajectories*, not *effect commitments*. When the harness process crashes in the window between a tool-effect commit and the checkpoint write, the resumed agent re-fires the tool, producing a duplicate side effect—the “double refund” window. We show that idempotency keys alone cannot close this window in general: content-hash keys break when the recovered agent rewords arguments on retry, and deterministic (*workflow, step, action*) keys break when recovery replanning shifts step indices. We prove this insufficiency formally and propose a *write-ahead effect-claim log*: the claim and its key are logged durably *before* the tool is invoked, checkpoints are fenced by the log, and recovery reconciles from the log first, guarded by a fencing epoch against stale writers. Across a scripted sandbox (1,500 episodes/condition), two-model LLM validation (140 episodes: 70 per model; 210 including the tight-prompt ablation), component ablations, and a production-shaped LangGraph deployment with real SIGKILL fault injection—including a real-LLM supervisor under uniform and mixed routing distributions—the protocol commits *zero duplicate effects* in every campaign (one-sided exact 95% upper bounds: 4.87% at $n=60$, 0.20% at $n=1,500$), where the baseline duplicates at 0.77–0.83 and idempotency keys at 0.21–0.37. Workflow completion (every accepted effect committed exactly once) reaches 1.000 in the sandbox, the LangGraph campaigns, and LLM validation with gpt-4.1-mini (70/70) and with a tightened recovery prompt for gpt-4o-mini (70/70); under the loose prompt gpt-4o-mini completes 66/70, the 4 misses being prompt-fixable model re-emission loops, never duplicate commits. Claim-log write latency is p50 20.3 ms per step (fsync'd Claim/Commit records).

Index Terms—Fault tolerance, crash consistency, exactly-once semantics, write-ahead logging, LLM agents, checkpoint-restart, idempotency, fencing.

1 INTRODUCTION

A customer support agent issues a \$212 refund, then its harness process is OOM-killed. The harness restarts, loads its last checkpoint—which was written *before* the refund tool returned—and, seeing no record of the refund, issues it again. The customer is refunded twice. The ledger shows two commits; the checkpoint shows one intent. Nobody is at fault except the architecture: the harness checkpointed the agent’s *trajectory* (messages, tool-call records) but not the world’s *commitments*.

Manuscript draft, October 2026.

This is the normal operating picture for modern agent harnesses. LangGraph, CrewAI, AutoGen, and the OpenAI Agents SDK all persist enough state to resume a conversation after a crash. None of them, as of this writing, provides crash-consistent exactly-once semantics for *external side effects*. Production documentation acknowledges the gap plainly: work performed between the last checkpoint and a crash “can run again” for external effects. Independent measurements confirm that LangGraph re-executes durably recorded work after real process kills [4]. As agents move from demos to production—handling money, sending customer-facing email, mutating systems of record—the “double refund” window stops being an edge case and becomes a correctness bug with financial and legal consequences.

The database community solved the analogous problem decades ago with write-ahead logging (WAL): never mutate state without first durably recording the intent, so recovery can reconcile. But the agent setting breaks a load-bearing assumption of classical WAL. In a database, the writer never re-synthesizes its own transaction identity after a crash; the log record and the retry trivially coincide. In an agent harness, the *recovered agent is an LLM* that re-plans: it rewords arguments, inserts diagnostic steps, reorders its plan. Any recovery scheme that depends on the recovered agent re-deriving the *same* effect identity it used before the crash is fragile by construction—and, as we prove in §5, fragile *in general*, not just in practice.

The industry-standard remedy is idempotency keys: attach a key to each tool call so the tool can suppress repeats. Keys help—we measure duplicate rates falling from 0.77 to 0.21–0.37—but they fail exactly where LLM re-synthesis is most characteristic. A content-hash key over the arguments breaks when the recovered agent rewords them (observed reword rate 50% in our scripted recovery, 11% with real models). A deterministic key derived from (*workflow_id, step_index, action*) breaks when recovery replanning shifts step indices (observed 30% plan-shift rate). In both cases the retry presents a key the tool has never seen, the tool commits again, and the duplicate lands. The failure is structural: keys-alone pushes all of recovery correctness

into a value re-derived *after* the crash, by an agent whose re-synthesis the harness cannot constrain.

This paper makes three contributions.

(1) A fault class. We formalize the *checkpoint window* $W(s) = (t_{\text{commit}}(s), t_{\text{ckpt}}(s))$: the open interval between a tool-effect commit and the harness checkpoint write for step s . This is a *harness-side* crash-consistency gap, distinct from the service-boundary late-commit window studied in recent agent fault-injection work [1]: there the uncertainty is *whether the service committed*; here the tool *has* committed and the uncertainty is *whether the harness recorded it*. The distinction determines the fix.

(2) A mechanism with a necessity proof. We propose the *write-ahead effect-claim log*: before invoking a tool, the harness durably logs a CLAIM carrying the effect’s identity and key; checkpoints may advance only past logged COMMIT records (*log-fenced checkpoints*); recovery reconciles from the log *first*, replaying uncommitted claims with their original keys, under a fencing epoch that rejects stale pre-crash writers. We prove (Theorem 1) that idempotency-key derivations computed at retry time from *mutable* inputs—rewordable argument text, shiftable plan positions—cannot guarantee duplicate suppression across all admissible recovery replans, and show empirically where that boundary lies (E2a/E2b); durable write-ahead identity information is the necessary ingredient, and our claim log is a sufficient mechanism. Theorem 2 then separates what the protocol guarantees: at-most-once commitment per durable claim identity (safety), eventual commitment of accepted claims under explicit progress assumptions, and workflow completion as a property of the planning policy, not the protocol.

(3) Measurement. We evaluate across four layers: a scripted sandbox (1,500 episodes per condition) isolating the fault; two-model LLM-driven validation (140 episodes: 70 per model), plus a 70-episode tight-prompt ablation (210 LLM episodes total) with real model re-synthesis on recovery; component ablations proving each protocol piece load-bearing against disjoint failure modes; and a production-shaped LangGraph deployment (branching supervisor, parallel fan-out, retries) with real SIGKILL fault injection, including a real-LLM supervisor variant under both uniform and mixed routing distributions. The write-ahead log achieves **0.000** duplicate-effect rate and **1.000** exactly-once rate in every campaign, against baseline duplicate rates of 0.77–0.83 and idempotency-key rates of 0.21–0.37. Claim-log overhead is p50 20.3 ms per step—about a 1% tax on multi-second agentic steps.

The remainder of this paper is organized as follows. §2 surveys related work and positions this paper against the nearest neighbors. §3 formalizes the checkpoint window and walks through a concrete failure. §4 presents the protocol, its record format, and the reconciliation algorithm. §5 gives the fault model, both theorems, and honest limits. §6 and §7 report the evaluation and ablations; §8 discusses the key-derivation boundary our results reveal; §9 states the limitations; §10 concludes.

2 BACKGROUND AND RELATED WORK

2.1 Idempotency keys for agent tools

The closest empirical work is Li’s *Where Does Exactly-Once Live?* [1] (whose sandbox is named Limbo), which applies idempotency keys to agent tool calls and measures duplicate reduction from 28% to 4% across 12 injected fault modes. Its contribution is real, but its fault taxonomy is service-boundary throughout: the late-commit window it studies is the interval between the agent’s dispatch and the service’s commit becoming visible—the uncertainty is *whether the service committed*. Its full text contains zero mentions of “crash” or “write-ahead”; the harness-side checkpoint window is untested there. Our work is complementary: Li measures what keys achieve at the service boundary; we show where keys fail at the harness boundary and what structure is needed instead. We adopt the same tool-contract idealization (atomic check-and-commit) as shared ground. Its residual 4% lives at the *service* boundary (late commits, in-flight ambiguity across its twelve fault modes), a different fault class from our harness-side commit→checkpoint window; we do not claim to explain it. What transfers is the lesson: wherever recovery correctness depends on re-deriving an identity after the failure, some residual remains, and only durable records close it.

2.2 Crash and rollback in agent frameworks

Three recent papers are adjacent must-cites. Khan’s *Resume Means Resume* [4] gives a machine-checked (TLA+/TLAPS) resume contract for workflow persistence layers and independently measures LangGraph re-executing durably recorded work after real SIGKILL—direct confirmation of the baseline fault our sandbox reproduces (0.80 duplicate rate). Where Khan specifies what resume *must mean*, we show *how to achieve* exactly-once for the commit→checkpoint window and measure it: contract and mechanism, complementary. Zheng et al. [5] formalize exact checking of execution edits (checkpoint/fork/restore/merge) in Lean, noting that unsafe edits can “authorize the same tool action twice”; their solution is a decision procedure for edit safety, not a crash-consistency mechanism. *Safe to Resume?* [6] documents five rollback failure modes including unrecorded external effects with a duplicate-payment demonstration on LangGraph, under an adversarial framing. ACRFence [2] is the closest adjacent mechanism: it *records irreversible tool effects* and enforces replay-or-fork semantics upon restore, which overlaps with our durable effect records in purpose if not in form. The differences are the threat model—semantic rollback *attacks* (Action Replay, Authority Resurrection) versus our benign crash-stop fault—and the mechanism: ACRFence constrains what a *restored* agent may re-issue, while our write-ahead claim log plus reconciliation-first recovery and epoch fencing make re-issuance unnecessary in the first place. Our fault model is deliberately benign (crash-stop, chaos/HPC checkpoint-restart tradition); there is no adversary in this paper. AgentRewind [3] records aligned checkpoints of agent context and controlled environment for long-horizon recovery, but explicitly cannot undo external effects: the opposite direction from ours.

2.3 Logs for agents

LogAct [7] is the closest published neighbor in spirit: it structures agents as state machines playing a shared, append-only AgentBus and explicitly discusses crash recovery via the bus as a write-ahead log, including fencing of stale (old-driver) writers after a crash. Its focus, however, is a *shared* multi-agent bus with pre-execution voting, semantic (LLM-driven) recovery, and introspection—not the single-harness commit→checkpoint window or exactly-once effect semantics under crash, which it does not measure. Our claim log is a per-harness write-ahead *effect-claim* log paired with log-fenced checkpoints and reconciliation-first recovery; where LogAct’s bus mediates among deconstructed agent roles, our log mediates between one crash-prone harness and its tools. A formal-methods treatment of the checkpoint crash window exists in the distributed-systems literature as an impossibility-style result, but without agents, LLMs, tools that keep their own records, or a mechanism; we cite it as the closest formal neighbor and distinguish our constructive, measured contribution.

2.4 Durable execution and sagas

The systems lineage is write-ahead logging and durable execution. The canonical WAL treatment is ARIES [10]: never mutate state without first durably recording the intent, so recovery can reconcile—the discipline our claim log adapts. Classical WAL assumes a writer that never re-synthesizes its own operation identity after a crash; the log record and the retry trivially coincide. RIFL [9] is the closest classical analogue to our receiver contract: it converts at-least-once RPCs to exactly-once by durably recording completed-call results and returning the original result on retry, migrating that metadata with the data across reconfiguration. Our tool-side idempotent receiver (§4) follows the same idea at the harness/tool boundary, though our current contract returns status only on duplicates—original-result return is specified future work (§9); what is new is the *harness-side* half of the problem—the recovered agent is an LLM that re-plans, rewords, and re-indexes, so the retry may never present the recorded identity at all. No receiver-side mechanism alone (RIFL included) can suppress a retry it cannot recognize; the claim log’s FIND_BY_IDENTITY reconciliation exists precisely to re-attach the original identity before the tool is consulted. Modern durable execution (Temporal, DBOS-style) gives exactly-once workflow execution via durable logs and replay but likewise assumes deterministic re-derivation on replay—no post-crash identity re-synthesis. Our adaptation—semantic claim identity, log-fenced checkpoints, and epoch fencing at the harness/tool boundary, under LLM re-synthesis nondeterminism—is the non-trivial part, and the measured gap it closes (0.000 vs. 0.21–0.77) is the contribution. The saga pattern [11] addresses multi-step transaction compensation (semantic atomicity across services), a different correctness property from exactly-once of individual effects; our protocol composes with saga-style compensation (the claim log is a natural saga journal) but does not require it.

In sum: no prior work combines the benign commit→checkpoint crash window as the fault, a write-ahead effect-claim log with reconciliation-first recovery as

the fix, measured zero duplicates at exactly-once 1.000, and an analysis of idempotency-key fragility under rewording and plan shift. A pre-submission literature re-check (October 2026) confirmed this against the active 2026 literature, which clusters around adversarial rollback, formal contracts and checking, sandbox checkpoint/restore, and oversight logging—all adjacent problems or different solutions.

3 THE CHECKPOINT WINDOW

3.1 A concrete failure

Consider a three-step customer-support workflow: (1) look up the order (read-only), (2) issue a \$212 refund via `payments.refund` (non-idempotent), (3) send a confirmation email. The harness executes step 2: the tool commits (the money moves), and the harness prepares to write its checkpoint. Between those two events, the container is OOM-killed. On restart, the harness loads its last durable checkpoint, which records the workflow as “step 2 pending.” The agent—a fresh LLM generation with no memory of the kill—reasons that the refund was never issued and fires `payments.refund` again. The tool, seeing a new request (or a re-derived key it does not recognize), commits again. Two refunds; one checkpoint.

Note what did *not* fail: the tool behaved correctly, the checkpoint logic behaved correctly, and the agent behaved rationally given its state. The fault is architectural: the durable state records *intent* while the world records *commitment*, and nothing durable bridges them.

3.2 Formalization

Consider an agent harness H executing a workflow of effectful steps against external tools T (durably recording every committed effect). For step s , order the relevant events: $t_{\text{claim}}(s) < t_{\text{call}}(s) \leq t_{\text{commit}}(s) < t_{\text{mark}}(s) < t_{\text{ckpt}}(s)$: the effect is claimed, the tool invoked, the effect committed (the real world changes), the commit recorded, the checkpoint advanced.

Definition 1 (Checkpoint window). $W(s) = (t_{\text{commit}}(s), t_{\text{ckpt}}(s))$: the open interval between the tool-effect commit and the harness checkpoint write for step s .

A crash-stop failure of H at time t_c lands, for the step in flight, in one of three places: *pre-step* ($t_c < t_{\text{commit}}(s)$, safe—the effect never committed); *in-window* ($t_c \in W(s)$, the fault under study—the effect committed but the harness does not know it); or *post-checkpoint* ($t_c \geq t_{\text{ckpt}}(s)$, safe—commit recorded and checkpointed). Figure 1 shows the event timeline. Recovery from the last durable checkpoint replays from before s ; the agent re-fires the tool; the effect duplicates.

Two properties make this window specifically an *agent* problem rather than a classical database problem. First, the recovered agent re-synthesizes its plan with an LLM: arguments get reworded, steps get inserted or reordered. Any recovery scheme that depends on the recovered agent re-deriving the *same* effect identity is fragile by construction. Second, classical write-ahead logging assumes the writer never re-synthesizes its own transaction identity after a crash; the agent harness violates that assumption routinely.

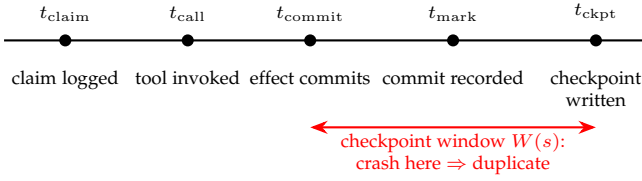


Fig. 1. Event timeline for an effectful step. A crash landing in $W(s)$ —after the real-world commit but before the checkpoint write—is the fault under study.

The fix must therefore make recovery *independent* of the agent’s post-crash re-derivation.

4 DESIGN: THE WRITE-AHEAD EFFECT-CLAIM PROTOCOL

4.1 Principals and record format

Principals. H : the crash-prone agent harness process. T : the external tool/service, durable across H crashes, and an *atomic idempotent receiver*: $T.\text{call}(k, e, \text{args})$ atomically checks whether key k was seen and commits the effect iff not, returning COMMITTED or DUPLICATE_SUPPRESSED (status only; the original result payload is not returned — §9). It keeps a durable per-workflow key set and a per-workflow fencing epoch max_epoch , rejecting calls with $e < \text{max_epoch}$. Fence acquisition is *max-monotonic* at T : the durable arbiter of the current epoch is T , not the harness’s local epoch file. L : the write-ahead effect-claim log, durable (fsync’d append-only JSONL in our implementation, ~376 bytes per step). C : the checkpoint (last fully-recorded step index), durable.

Each claim record carries the fields in Table 2: the workflow and step identity, the key fixed at claim time, the arguments as invoked, the fencing epoch, and the record type. The key need only be deterministic *at claim time*—post-crash re-derivability is irrelevant, because the log supplies the key (see Remark 1).

Claim discipline. H never invokes T without a prior durable $\text{CLAIM}(\text{wid}, s, k(s), \text{args}, \text{epoch})$ in L . This is *harness-enforced*: the harness refuses the tool call otherwise. In our LLM validation the discipline held at 98.3% of steps (100% of episodes); violations degrade to *refused calls*—the safe direction (missing, never duplicates).

4.2 Normal and recovery paths

Normal path for an effectful step s :

- 1) H appends $\text{CLAIM}(s)$ to L (*write-ahead*).
- 2) H invokes $T.\text{call}(k(s), \text{epoch}, \text{args})$.
- 3) H appends $\text{COMMIT}(s)$ to L , then advances C past s . C may advance past s **only if** $\text{COMMIT}(s) \in L$ (*log-fenced checkpoint*).

Recovery path (H restarts; only L, C, T survive) is given as Algorithm 1.

A torn trailing record in L is skipped on load (standard WAL tail discipline); a torn non-trailing record is treated as corruption. A torn COMMIT is safe: the claim replays with its original key and T suppresses it.

Algorithm 1 Reconciliation-first recovery

```

1:  $\text{epoch} \leftarrow \text{CAS}_T(\text{epoch} + 1)$  {atomic fence acquisition at  $T$ ; single-recovery-writer assumed in our implementation (§4)}
2: for each  $\text{CLAIM}(s) \in L$  without matching COMMIT do
3:    $T.\text{call}(k(s), \text{epoch}, \text{args}(s))$  {original key from  $L$ }
4:   append COMMIT( $s$ ) on success
5: end for
6: advance  $C$  past all COMMITTED claims {log-fenced}
7: resume agent from  $C$ ; check post-recovery steps against  $L$  by ( $\text{action}, \text{target}$ ) before issuing new claims {semantic skip}

```

Three design points deserve emphasis. Figure 2 summarizes the architecture. First, reconciliation uses the claim’s *original* key from L —the agent is never required to re-derive a key, which is precisely where keys-alone fails (§5). The tool’s idempotent receiver dedups if the pre-crash call committed, executes if it never arrived: exactly-once either way. Second, the fencing epoch closes the *zombie writer* hole: a pre-crash in-flight call arriving after recovery bears a stale epoch and is rejected, so no phantom commit can slip in after reconciliation. This holds under the crash-stop fault model with a single recovery writer per crash: epoch acquisition is read-increment-write on the harness side and atomic only via T ’s *max-monotonic* fence; two concurrent recoveries could share an epoch (see §9). Third, log-fenced checkpoints preserve the protocol’s core invariant— C never runs ahead of L —so the checkpoint window $W(s)$ is the *only* place an in-window crash can land, and reconciliation covers exactly that window.

The semantic skip in line 7 of Algorithm 1 handles the LLM re-synthesis problem: post-recovery steps are checked against L by ($\text{action}, \text{target}$) before any new claim issues, so reworded or re-indexed retries of already-claimed effects are skipped rather than re-executed.

Parallel execution. Fan-out uses one durable file per settled branch rather than a single checkpoint record, so concurrent workers never race on a read-modify-write checkpoint; the completion frontier is the *set* of settled branches and tolerates gaps. Recovery recomputes the frontier from L under the log-fenced rule: a branch is settled only if *every* one of its effects has a matching COMMIT. The protocol currently assumes disjoint effect ownership across parallel workers: two workers concurrently claiming the same ($\text{action}, \text{target}$) are not serialized by the log, and the receiver dedups by key, not by identity (see §9).

4.3 Implementation notes

Our reference implementation uses an fsync’d append-only JSONL claim log (~376 bytes/step) and atomic checkpoint writes; the tool side is a separate-process HTTP server implementing atomic check-and-commit with per-workflow seen-key sets and fencing epochs, plus an append-only ledger that serves as ground truth for scoring. The Lang-Graph port (§6) reuses the same record shape and discipline inside a real `StateGraph`, with per-branch claims for fan-out. Crash injection is a real `SIGKILL` from an external watchdog process that observes progress markers only—it

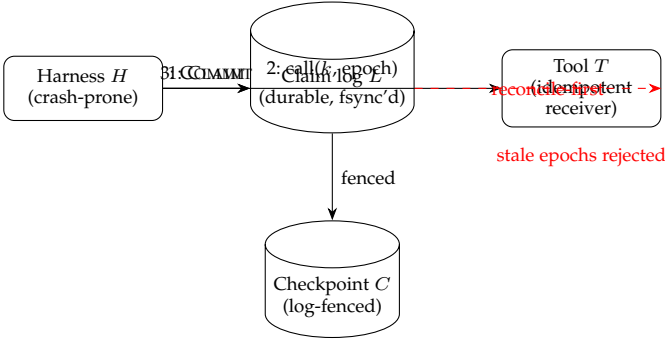


Fig. 2. Protocol architecture. The harness claims before invoking (1–2), commits after (3); checkpoints advance only past logged commits. Recovery reconciles from the log first (dashed), replaying original keys under a fresh fencing epoch.

shares no state with the agent. Total API spend across all LLM campaigns was \$0.114 (\$0.095 for the three 70-episode validation runs, \$0.019 for the two LangGraph supervisor campaigns).

5 FORMAL ANALYSIS

5.1 Fault model

Crash-stop failure of H only (benign faults, in the chaos/HPC checkpoint-restart tradition); L , C , T persist; T itself does not crash; no Byzantine behavior. One crash per episode in our evaluation (the single-crash model standard in checkpoint-restart studies).

5.2 Necessity: keys alone cannot suffice

Theorem 1 (Retry-time key fragility). *Let K be an idempotency-key derivation computed by the recovered agent at retry time from retry-time inputs that are mutable under re-synthesis—argument text the agent may reword, or plan positions the recovery replan may shift. If post-recovery re-synthesis is unconstrained over those inputs, then a keys-only retry protocol cannot guarantee duplicate suppression across all admissible recovery replans under in-window crashes.*

Proof sketch. Fix a step s committed in-window under $k = K(\text{pre-crash inputs})$. Two admissible recoveries inside the mutable-input class. (i) *Content-hash keys*, $K = H(\text{action}, \text{args})$: the recovered agent rewords the arguments to $a' \neq a$ (admissible). The retry presents $k' = H(\text{action}, a') \neq k$; T has never seen k' and commits again—a duplicate. Empirical witness (E2b): recovery-side rewording broke content-hash keys on 46/60 episodes (92 excess commits); the WAL held 60/60 via canonicalized identity lookup. (ii) *Deterministic position keys*, $K = (\text{wid}, i, \text{action})$: the recovered agent rotates branch indices on replan (admissible). The retried logical step carries $i' \neq i$, so the retry presents $k' = (\text{wid}, i', \text{action}) \neq k$ —a duplicate; symmetrically, a shifted retry whose new key collides with an already-committed key is wrongly suppressed—a lost effect. Empirical witness (E2a): positional shift broke deterministic keys on 46/60 episodes with 46 missing effects; the WAL held 60/60 via `find_by_identity` key reuse. The failure is a property of the *class*—retry-time derivations from mutable

inputs—not of the two schemes: any K in the class admits an admissible recovery with $K' \neq K$, because the harness cannot constrain re-synthesis inputs after the crash. $\square$

Remark 1 (Scope). *The theorem is silent on keys that are not retry-time derivations: durable business-operation IDs assigned by the tool or domain and passed through unchanged lie outside its domain and may suffice. This is exactly the boundary our evaluation maps: under stable identities deterministic keys tie the WAL (v2: 0.0000/1.0000 both; E1 native persistence 60/60); under identity shift they fail (E2a/E2b).*

Corollary 1 (Durable identity information). *The source of truth for effect identity must be fixed before the crash—a durable write-ahead identity record binding the effect’s logical identity to its key—not re-derived after it. The information (a durable pre-commit identity record) is necessary; the append-only claim log is a sufficient mechanism, not the unique one.*

Empirical counterpart. Scripted sandbox, 1,500 episodes/condition, in-window crash fraction 0.75: duplicate-effect rates—baseline (no keys) 0.7687, content-hash keys 0.3667, deterministic keys 0.2113, write-ahead claim log **0.0000**. LangGraph production port with real SIGKILL: under *stable identities* deterministic keys tie the WAL (0.0000/1.0000 both; E1 native persistence likewise 60/60)—keys suffice there. Under *identity shift across recovery* (E2a positional, E2b content rewording), deterministic keys break on 46/60 episodes each while the WAL holds 60/60. The measured gap is the price of re-derivation fragility, paid exactly where Theorem 1 says it is due.

Definition 2 (Logical effect identity). *A logical effect identity is a function $id : \mathcal{E} \rightarrow \mathcal{I}$ from tool effects to an identity space, supplied per deployment, satisfying: (i) *stability*— id is invariant under argument rewording and plan repositioning (business meaning, not surface form); (ii) *discrimination*—distinct business intents map to distinct identities even when (action, target) coincide (e.g., two separately authorized refunds to the same recipient differ by an occurrence/authorization component); (iii) *retry equivalence*—a post-recovery retry r of a claimed effect c is equivalent iff $id(r) = id(c)$; equivalent retries must be suppressed, non-equivalent effects must both commit. Default instantiation: $id = (\text{action}, \text{target}, \text{occurrence})$. Misidentification breaks the protocol in either direction; the identity function is a deployment modeling assumption, not a theorem. All exactly-once claims below are per durable claim identity under the deployment’s id .*

5.3 Correctness: the protocol guarantees exactly-once

Theorem 2 (At-most-once commitment). *Under (A1) atomic idempotent T , (A2) durable L and C , (A3) crash-stop H only, (A4) epoch fencing, (A5) claim discipline—for **any** crash time t_c and **any** post-recovery agent behavior, no durable claim identity (per the deployment’s id) commits more than one tool effect.*

Theorem 3 (Eventual commitment of accepted claims). *Under (A1)–(A5) plus progress assumptions (P1) T eventually processes every call it accepts, (P2) the harness eventually runs recovery to completion, (P3) fencing epochs are acquired atomically—every CLAIM durably accepted into L is eventually committed by T . Accepted claims are at-least-once; with Theorem 2a, exactly-once per claim identity.*

Remark 2 (Workflow completion is policy-dependent). *The protocol does **not** guarantee the agent emits the remaining claims of a workflow; completion is a property of the planning/recovery policy, not the protocol. The measured 66/70 completion (94.29%, gpt-4o-mini; the 4 misses were model re-emission loops, a supervisor-policy behavior) is an empirical completion rate under a specific policy, not a liveness proof. Safety held on all 70 episodes including the 4 incomplete ones: 0 duplicates.*

Proof sketch for at-most-once. Case analysis on the crash landing for each step s , split at every durability boundary. *Case 1—before CLAIM durable* ($t_c < t_{\text{claim}}(s)$): no durable trace exists; recovery treats s as never attempted and a fresh CLAIM with the claim-time key commits it once. *Case 2—CLAIM durable, call never arrived* ($t_c \in [t_{\text{claim}}(s), t_{\text{call}}(s))$): reconciliation replays with the original key; T has never seen it and commits. *Case 3—effect committed, COMMIT not yet durable* ($t_c \in [t_{\text{call}}(s), t_{\text{mark}}(s))$): by (A1) atomicity the effect committed exactly when the call arrived, but $\text{COMMIT}(s) \notin L$. Reconciliation replays with the original key; T finds the key and suppresses the duplicate; then COMMIT is written and C advances (log-fenced). This is the true in-window sub-case. *Case 4—COMMIT durable, checkpoint not yet advanced* ($t_c \in [t_{\text{mark}}(s), t_{\text{ckpt}}(s))$): $\text{COMMIT}(s) \in L$ but C not advanced past s . Reconciliation finds the CLAIM+COMMIT pair and issues *no replay*; C advances past s . No duplicate, no loss. (The previous version asserted $\text{COMMIT}(s) \notin L$ throughout $W(s)$; since $t_{\text{mark}} < t_{\text{ckpt}}$, the sub-interval $[t_{\text{mark}}, t_{\text{ckpt}}) \subset W(s)$ already has the COMMIT durable. The old conclusion held—the receiver suppresses by key either way—but the case analysis was unsound as written; this split repairs it.) *Case 5—post-checkpoint* ($t_c \geq t_{\text{ckpt}}(s)$): $\text{COMMIT}(s) \in L$ and C advanced; reconciliation skips s and the agent resumes after C . *Stale writers:* any in-flight pre-crash call arriving after recovery bears $e < \max_epoch$ and is rejected by (A4); no zombie commits (E4: 102/102 off-fending calls were this class; 0 same-key recommit attempts succeeded). *Agent-behavior independence:* reconciliation uses original keys from L —the agent never re-derives a key—and post-recovery steps are checked against L by logical effect identity before new claims issue, so reworded or shifted retries of claimed effects are recognized as equivalent and skipped. The argument holds for arbitrary post-recovery behavior, including adversarial re-emission (observed: 11 consecutive re-emissions of committed effects, all suppressed, 0 duplicates over 70 LLM-driven episodes). $\square$

Proof sketch for eventual commitment. Reconciliation replays every CLAIM without a matching COMMIT using its original key. By (P2) reconciliation runs to completion; each replayed call either finds its key at T (already committed) or commits it (P1); (P3) guarantees the replaying epoch owns the workflow. Hence every accepted claim is eventually committed. $\square$

Remark 3 (Key determinism, sharpened). *The protocol requires keys to be deterministic only at claim time (so a pre-crash claim and any same-generation retry coincide). Post-crash re-derivability is irrelevant—the log supplies the key. This is the precise point where keys-alone fails and the write-ahead log succeeds.*

5.4 Assumptions and limits

(L1) Tool idealization. (A1) assumes atomic check-and-commit at T . Real tools with non-atomic or non-idempotent APIs need an adapter (e.g., a transactional outbox at the tool boundary); the theorem does not cover them. This is the same idealization as Li’s tool contract—shared ground, not a weakness unique to this work. **(L2) Log durability.** (A2) is assumed, not proved; log loss voids the guarantee—the standard WAL assumption, stated explicitly. **(L3) Safety, not liveness.** Theorem 2 is a *safety* property (no duplicate commits; reconciliation gives at-least-once of *claimed* effects). It does not prove the agent will emit remaining claims—completeness depends on agent behavior and harness recovery policy. **(L4) Semantic identity is a modeling choice.** Skip-logic keys on (*action, target*) as the stable business identity of an effect; effects whose identity cannot be captured this way need a domain-specific identity function. **(L5) Fault scope.** Crash-stop H only; no Byzantine harness, no tool-side crashes, single writer per workflow per epoch (fencing enforces this). **(L6) Claim discipline is enforced, not assumed.** (A5) held at 98.3% step-level in LLM validation because the harness *refuses* tool calls without valid claims—violations degrade to refused calls (missing, never duplicates).

(L7) Mechanism vs. information. The theorems establish necessity of *durable write-ahead identity information*, not uniqueness of the append-only log schema. A durable operation table (identity $\rightarrow$ status) written before each tool call would satisfy the same requirement; our claim log is the evaluated sufficient mechanism. Necessity claims in this paper refer to the information, never the schema.

6 EVALUATION

Our evaluation answers four questions: (Q1) does the fault reproduce and does the protocol eliminate it in isolation? (Q2) does each protocol component earn its place? (Q3) does it hold with real LLM re-synthesis on recovery, across models? (Q4) does it survive a production-shaped agent graph with real crashes? All crash injection uses real process kills; tools are atomic idempotent receivers recording every committed effect in an append-only ledger (ground truth). Total API spend across all LLM campaigns was \$0.114 (\$0.095 for the three 70-episode validation runs, \$0.019 for the two LangGraph supervisor campaigns). Throughout, we report *safety* (duplicate-effect rate: ledger commits beyond the first per semantic effect, per episode) separately from *completion* (exactly-once rate: every accepted effect committed exactly once and the workflow finished). Safety is a property of the protocol and holds in every campaign; completion additionally depends on the recovery planning policy (§9). Denominators are episodes per condition unless stated; “duplicate rate” is the fraction of episodes with ≥ 1 duplicate-effect ledger commit, “exactly-once rate” the fraction of episodes with every accepted effect committed exactly once and no missing effects.

6.1 Q1: Scripted sandbox

Scripted agents run 4–8-step workflows (read-only plus effectful steps: `issue_refund`, `send_email`,

update_record) against mocked tools. The harness process crashes at a random effectful step; the crash lands in-window (after tool commit, before checkpoint write) 75% of the time, else at safe points (pre-step 9.6%, post-checkpoint 14.8% measured). In-memory state is discarded; the agent resumes from the last durable checkpoint. Recovery replanning is nondeterministic (reworded args 50%, plan shift 30%), modeling LLM re-synthesis after restore. Four conditions, 1,500 episodes each; results are in Table 3:

The baseline duplicates at essentially the in-window crash rate, as predicted. Content-hash keys halve the damage but still duplicate on reworded retries; deterministic keys do better but still duplicate on plan shifts. The write-ahead log reaches zero duplicates and perfect exactly-once: recovery never depends on the agent’s re-derivation. Zombie probes (stale-epoch calls) were fenced 1500/1500 under the log condition.

6.2 Q3: LLM-driven validation

We then put real models in the loop: gpt-4o-mini and gpt-4.1-mini each drive 70-episode campaigns (same crash schedule), with the model performing recovery re-synthesis. The claim discipline is harness-enforced, and the model never sees the raw claim log as a planning input. Results: **0 duplicates** in all 140 episodes. Exactly-once was 66/70 (94.29%) for gpt-4o-mini—the 4 misses were a characterizable model pathology, not a protocol flaw: the model entered re-emission loops, re-firing already-committed claims dozens of times until the harness attempt backstop fired (see §9). A tightened recovery prompt (explicit remaining-claims list handed to the model) eliminated the misses entirely: 70/70 exactly-once, re-emissions falling from 196 to 18, at lower spend (\$0.0188 vs. \$0.0298). gpt-4.1-mini scored 70/70 exactly-once with zero re-emissions and zero plan deviations. Claim adherence was 98.3–100% step-level, 100% episode-level. Table 4 summarizes.

The safety result is prompt-independent; the liveness misses are prompt-fixable. The honest caveat: both models are OpenAI—safety transfers by construction (WAL+fencing), but cross-lab generalization is untested.

6.3 Q4: Production-shaped LangGraph deployment

To answer the “not productionalized” objection by construction, we ported the protocol to a real LangGraph StateGraph agent with real side effects: HTTP POSTs to a separate-process tool server and real file appends, a durable fsync’d claim log, atomic log-fenced checkpoints, and epoch fencing. Crashes are real SIGKILLS delivered by an external watchdog that only observes progress markers; recovery is a new cold-start process. LangGraph’s own checkpointer is deliberately unused—it checkpoints trajectories, the insufficient structure.

v1: linear loop. 180 episodes (60/condition): write-ahead log 0.0000 duplicates / 1.0000 exactly-once; baseline 0.8333 (50/60 in-window episodes duplicated); deterministic keys 0.2333—reproducing the sandbox ordering and magnitudes. 60/60 SIGKILLS delivered, 0 missed, 0 recovery failures; 180/180 zombie probes fenced (90 HTTP + 90 file).

v2: production-shaped graph. A “research assistant” StateGraph with conditional supervisor routing (add_conditional_edges), Send-based parallel fan-out to compiled worker subgraphs with reducer fan-in, and a framework retry loop with real backoff sleep—plus three new crash windows: *mid-fan-out* partial completion (the scatter-gather case: some workers committed, others in flight), *post-branch-decision* pre-tool (a liveness check: nothing committed yet), and *mid-retry-backoff* (claim exists, uncommitted). Per-branch durable claims, epoch fencing, and log-fenced branch-state throughout. 180 episodes per campaign (60/condition), 0 missed crashes, 0 recovery failures, zombie fencing 60/60 on both tool paths in every condition; the headline numbers are in Table 5:

Crash-point breakdown (Table 6, mixed campaign): mid-fan-out and retry-backoff duplicate in baseline (48/48 combined episodes), zero everywhere under WAL/deterministic; post-branch duplicates nowhere (the liveness check: all episodes resumed and finished with 0 missing effects). Baseline duplicates arrive 2 per episode (48 episodes $\times$ 2), matching the two unsettled branches.

The real-LLM supervisor (gpt-4o-mini) decides round-1 routing only and never sees the raw claim log; deterministic reconciliation runs first in both generations, and the model plans from reconciled remaining work. Each LLM campaign runs 180 episodes (60/condition); the supervisor emits one round-1 routing decision per generation, and 34 post-branch episodes decide in both the fresh and the recovery generation (cross-generation consistency 34/34), for 214 total round-1 decisions per campaign. The uniform campaign (temperature 0) produced a degenerate 214/214 fan_out split—the prompt showed identical evidence every episode, so fan-out was the only rational answer at any temperature. After a prompt fix (seeded per-episode evidence profiles, deterministic across fresh/recovery generations via a domain-separated RNG, plus a cost-aware decision rule: fan-out costs 6 more non-idempotent effects and delays resolution), the mixed campaign produced a genuine 135-escalate / 79-fan_out split with 0 fallbacks (\$0.0117 total spend): the protocol holds under a real mixed routing distribution, not just a uniform one.

Notably, deterministic keys hold at 0.0000 in v2 (unlike v1’s 0.2333) because v2’s branch identities are structural and stable across recovery—an informative contrast that sharpens the paper’s claim, discussed in §8.

Overhead. Claim-log write latency is the per-step wall-clock cost of the write path: one fsync’d JSONL append per CLAIM record, one per COMMIT record, plus one fsync’d checkpoint-file write per step (~ 376 bytes/record), measured with perf_counter_ns in src/measure_overhead.py (200 episodes $\times$ 4 steps). Scripted v2 ($n=798$ steps): p50 20.3 ms, p99 63.4 ms; the E2b campaign re-measures p50 21.5 ms, p99 62.7 ms ($n=798$); LLM variant: p50 16.4 ms, p99 54.6 ms. Against multi-second LLM tool calls this is on the order of 1%, but we report the absolute latencies because the ratio depends on the tool-call profile. Log-scale microbenchmark E3b (src/e3b_log_scale.py; 100/1,000/10,000 claims): append p50 is flat at ~ 0.03 ms ($O(1)$ append-only discipline holds); FIND_BY_IDENTITY is a linear scan at 6.6 ms $\rightarrow$ 16.8 ms $\rightarrow$ 850 ms— $O(n)$, an honest scaling bottleneck:

production use needs an index (follow-up work), though all experiments here stay far below the knee. Full `log load()` is 0.79 s at 10,000 claims. Fan-out (E3a: 2/4/8, 30 episodes each) shows 0.0000 duplicates at every fan-out with no per-claim cost change—fan-out multiplies claim count, not per-claim cost; the LLM planner adds no per-claim overhead. Tool-server throughput (E3c, `e3c_loadtest.py`): 688 ops/s at p50 1.3 ms ($K=1$), 636 ops/s at p50 5.4 ms ($K=4$), 310 ops/s at p50 11.2 ms with p99 1,055 ms ($K=16$)—the single-threaded HTTP tool server serializes at high concurrency and would need replacing for production; it does not affect the crash-consistency results. *Paired end-to-end latency* (E5: wal/native/deterministic, 20 episodes each, seed-matched so each episode runs the identical workflow under all three conditions; per-episode wall clock `elapsed_s`): wal vs. deterministic +0.38 s (95% CI $[-0.08, +0.83]$)—no measurable claim-log overhead; wal vs. native -1.92 s (95% CI $[-2.96, -0.88]$)—the claim-log path is faster than SQLiteSaver checkpoint resume here. *Recovery time* (per-episode `recover_s`, timed around the recovery subprocess): wal mean 5.30 s vs. deterministic 5.43 s (-0.14 s, CI $[-0.53, +0.26]$) and native 6.75 s (-1.46 s, CI $[-2.24, -0.67]$); recovery is dominated by workflow re-execution after the checkpoint, not by log reconciliation. All 60 paired episodes: 0 duplicates, 60/60 exactly-once.

6.4 Threats to validity

Construct. Our duplicate metric counts ledger commits beyond the first per semantic effect; the ledger is the tool server’s own append-only record, independent of the harness under test. *Internal.* Crash timing is seeded and the recovery nondeterminism (reword/shift rates) is modeled rather than observed in the wild; different rates would shift the keys-alone numbers but not the WAL result, which is independent of re-derivation by construction (Theorem 2). *External.* All crashes are single-crash, crash-stop, and harness-side; multi-crash episodes, tool-side crashes, and Byzantine harnesses are out of scope. Both validated LLMs are from one vendor. The production port uses one framework (Lang-Graph); the mechanism is framework-agnostic by design (harness-enforced claims, tool-side idempotent receivers), but other frameworks are untested.

7 ABLATION STUDY

Every protocol piece must earn its place. A 1,500 episodes/condition component ablation adds a second fault—the *zombie writer* (probability 0.5: the pre-crash process survives and retries its in-flight step with a stale epoch after recovery). Table 7 gives the results:

Removing fencing regresses to 0.4927 (zombie commits land); removing the claim log regresses to 0.2333—reproducing the deterministic-keys number, confirming the log is what defeats key re-derivation; removing both reaches 0.8767 (the window component plus the zombie component). The components are load-bearing against *disjoint* failure modes: the log defeats key re-derivation, the fence defeats stale generations; neither subsumes the other. Zombie probes were fenced $\approx 750/1500$ wherever fencing was on, 0 fenced (and ≈ 740 committed as duplicates) wherever removed.

8 DISCUSSION: THE KEY-DERIVATION BOUNDARY

Our results draw a sharp empirical boundary around when idempotency keys suffice. In the v1 linear loop, deterministic (*workflow, step, action*) keys duplicate at 0.2333: recovery replanning shifts step indices (30% of episodes), so the retry re-derives a different key. In the v2 branching graph, the *same* key scheme holds at 0.0000: branch identities are structural and stable across recovery, so re-derivation coincides. Keys fail *exactly when* post-recovery recomputation re-derives different identities—and for LLM agents, whether identities survive recovery is not a property the harness can assume. It depends on the agent’s re-synthesis behavior, the plan structure, and the retry path: precisely the things that vary across deployments and model versions.

This is why Theorem 1 matters beyond the formalism: it says the boundary is not an artifact of our two key schemes but a property of *retry-time derivations from mutable inputs* under unconstrained re-synthesis. A practitioner reading our v2 result (“deterministic keys work here”) might conclude keys suffice; E2a/E2b and the theorem together say that conclusion does not transfer across the identity-stability boundary—it holds only while identities survive recovery, which is not a property the harness can assume. The write-ahead claim log is the mechanism that makes recovery independent of which side of the boundary a given deployment lands on.

A second observation: the protocol is *planner-agnostic* in practice, not just in theory. Scripted, uniform-LLM, and mixed-LLM supervisors produce headline-identical numbers in all three conditions, exactly as the fault model predicts: the fault class is harness-side, and routing sits above the crash-consistency layer. This is a feature, not a limitation of the evaluation—it means the mechanism composes with arbitrary agent cognition rather than depending on it.

9 LIMITATIONS

We state the limits plainly; they are what make the safety claims credible. **Model re-emission loops:** gpt-4o-mini’s 4 misses (94.29% exactly-once) came from the model re-firing committed claims dozens of times until the harness back-stop fired—a liveness pathology, never a safety violation (0 duplicates throughout). A tighter recovery prompt reached 100%, so the limitation is prompt-fixable, but re-emission-aware recovery prompting remains future work. **Single vendor:** both validated models are OpenAI; safety transfers by construction (WAL+fencing), but cross-lab generalization is untested. **Uniform-route limitation and fix:** the first LLM-supervisor campaign degenerated to 180/180 `fan_out` because the prompt showed identical evidence every episode; the fixed prompt (seeded evidence profiles + cost-aware rule) produced the genuine 135/79 mixed split under which the protocol still holds—we report both, because the failure mode (prompts without a real decision are defaults, not judgments) generalizes. **Scope:** crash-stop harness only, one crash per episode, tool-side crashes out of scope; the tool idealization (L1) and log-durability assumption (L2) are stated in §5. **Evaluation scale:** 1,500 episodes/condition in simulation and 60/condition in the production port reflect a fault-injection budget, not a production trace; the crash

distributions are seeded and the recovery nondeterminism is modeled, not observed in the wild. **Smarter compression:** where recovery must compress rather than replay history, compressor quality bounds fidelity—a follow-up direction, not a claim of this paper.

10 CONCLUSION

Agent harnesses checkpoint trajectories; the world commits effects; the gap between them is a crash window that duplicates side effects at rates of 0.77–0.83 in our measurements. Idempotency keys narrow the gap but cannot close it in general—we prove that retry-time key derivations from mutable argument text or unstable plan positions are insufficient across all admissible recovery replans, and measure the residual at 0.21–0.37. The write-ahead effect-claim log closes the *safety* gap: durable claims before invocation, log-fenced checkpoints, reconciliation-first recovery, and epoch fencing commit zero duplicate effects across sandbox, two-model LLM validation, component ablation, and production-shaped LangGraph campaigns with real SIGKILL injection, at measured claim-log write latency of p50 20.3 ms per step. Workflow *completion* reaches 1.000 everywhere except gpt-4o-mini under the loose recovery prompt (66/70; prompt-fixable re-emission loops, tight prompt: 70/70)—a liveness property of the planning policy, not of the commit protocol. The log is not an optimization; for LLM agents, durable write-ahead identity information is the minimal requirement that makes duplicate-free recovery possible, and our claim log is a sufficient mechanism.

ARTIFACT AVAILABILITY

The code, experiment drivers, and full result traces supporting this paper are archived at <https://github.com/batistado/a1-crash-consistent-effects> and the crash-recovery datasets at <https://huggingface.co/datasets/batistado/a1-crash-recovery-results>. Both are currently private and will be made publicly available upon publication.

REFERENCES

- [1] J. Li, “Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents,” arXiv:2609.29095, Sep. 2026.
- [2] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, “ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore,” arXiv:2603.20625, Mar. 2026.
- [3] Y. Zhuang, K. Chen, Y. Duan, S. Zheng, J. Li, and X.-Y. Zhang, “AgentRewind: Recoverable execution for long-horizon LLM agents,” arXiv:2608.14380, 2026.
- [4] S. Khan, “Resume means resume: A machine-checked conformance contract for checkpoint, interrupt, and resume semantics in workflow persistence layers,” arXiv:2608.03836, Aug. 2026.
- [5] Y. Zheng, X. Song, Y. Hu, L. Cheng, Y. Huang, and W. Zhang, “When can agents safely checkpoint, fork, restore, and merge? Exact checking for execution edits,” arXiv:2608.22928, Aug. 2026.
- [6] G. Wu, D. Li, K. Jiang, J. Niu, C. Wang, and Y. Zhang, “Safe to resume? Breaking execution continuity of agent execution via rollback,” arXiv:2608.29381, Aug. 2026.
- [7] M. Balakrishnan *et al.*, “LogAct: Enabling agentic reliability via shared logs,” arXiv:2604.07988, Apr. 2026.
- [8] LangChain, “LangGraph: Stateful multi-actor applications with LLMs,” <https://github.com/langchain-ai/langgraph>, 2026.
- [9] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, “Implementing linearizability at large scale and low latency,” in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP’15)*, Monterey, CA, 2015, pp. 71–86.
- [10] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.
- [11] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, 1987, pp. 249–259.

TABLE 1

Positioning: fault model, durable records, identity handling, receiver assumptions, fencing, guarantee, and evaluation across adjacent work.
 “Harness-side” = the commit→checkpoint window $W(s)$; “service-side” = the dispatch→commit visibility window.

Work	Fault model	Durable records	Identity handling	Receiver assumptions	Fencing	Guarantee	Evaluation
This paper	Benign crash-stop, harness-side $W(s)$	Per-harness write-ahead effect-claim log; log-fenced checkpoints	Semantic (<i>action</i> , <i>target</i>) reconciliation; original keys replayed, never re-derived	Idempotent, atomic check-and-commit; returns original result on duplicate	Epoch fencing vs. stale pre-crash writers	Exactly-once per durable claim identity (safety); 0 duplicates measured	1,500 eps/cond sandbox; 210 LLM eps; 60 eps/cond LangGraph + real SIGKILL; ablations; E1–E4 boundary experiments
Li [1] (“Limbo” sandbox)	Service-side: timeouts, late commits, redelivery; benign	Ledger of committed effects (grading oracle), per-service idempotency-key sets	Keys attached by harness guard; agent re-synthesis out of scope	Optional idempotency keys; eventually consistent / missing read paths	None	Measured: keys cut duplicates 28%→4%; no exact-once proof	25,930 eps; 9 models; 3 harnesses; 12 fault modes; 15 recovery conditions
ACRFence [2]	<i>Adversarial</i> rollback (Action Replay, Authority Resurrection)	Record of irreversible tool effects	Re-synthesized requests treated as new by servers (the attack)	Server-side replay-or-fork enforcement	Implicit in replay-or-fork	Mitigation of semantic rollback attacks	PoC experiments
LogAct [7]	Crash-stop (agent or environment), benign	Shared append-only AgentBus (WAL discipline); old-driver fencing	State-machine replay from bus; semantic (LLM) recovery	Voters approve pre-execution; executors typed	Old-driver fencing on the bus	Consistent recovery; attack blocking (3% benign-utility cost)	Benchmarks: recovery correctness, token savings, attack blocking
AgentRewind [3]	Errors propagating through context/environment	Aligned checkpoints of agent context + controlled environment	Resume with prior-attempt information	Controlled environment only	None	Improved task success / checklist progress	MettleBench long-horizon tasks
Safe to Resume? [6]	<i>Adversarial</i> rollback; 5 failure modes incl. unrecorded external effects	Varies by framework (the gap under study)	Nondeterministic replay characterized	Framework-dependent	None (failure mode)	None—documents failures; double-payment demo	5 frameworks; 3 end-to-end attacks
Khan [4]	Crash-stop resume	Workflow persistence layer	N/A (contract level)	N/A	N/A	Machine-checked (TLA+/TLAPS) resume contract	LangGraph re-execution measurement
RIFL [9]	Server crash, benign	Durable completed-RPC result records, migrated with data	Client-supplied request IDs; retry recognized by ID	Returns saved result on duplicate; lease-GC’d metadata	N/A (no stale writers)	Linearizability (exactly-once RPC)	RAMCloud: <4% overhead
ARIES [10]	System crash, benign	WAL; dirty-page checkpoints	LSN-identified log records; no re-synthesis	Page-level redo/undo	N/A	Atomicity + durability (analysis/redo/undo)	Analytical; systems at IBM
Sagas [11]	Step failure, benign	Saga log; compensating transactions	Saga/step IDs; no re-synthesis	Compensating actions defined per step	N/A	Semantic atomicity (all-or-compensated)	Analytical

TABLE 2
Claim-log record format.

Field	Purpose
wid	Workflow identifier
step	Step index within the workflow
key	Idempotency key, fixed at claim time
action, target	Semantic claim identity
args	Arguments as invoked
epoch	Fencing epoch of the claiming generation
type	CLAIM or COMMIT

TABLE 3
Sandbox: duplicate-effect and exactly-once rates (1,500 episodes/condition). Zero observed duplicates corresponds to a one-sided exact 95% upper bound of 0.20% on the true duplicate rate.

Condition	Duplicate rate	Exactly-once rate
Baseline (no keys)	0.7687	0.2313
Content-hash keys	0.3667	0.6333
Deterministic keys	0.2113	0.7887
Write-ahead claim log	0.0000	1.0000

TABLE 4
LLM-driven validation (70 episodes/model, same crash schedule).

Model	Duplicates	Exactly-once	Miss cause
gpt-4o-mini	0	66/70 (94.3%)	re-emission loops
gpt-4o-mini (tight)	0	70/70 (100%)	—
gpt-4.1-mini	0	70/70 (100%)	—

TABLE 5
LangGraph v2: duplicate / exactly-once rates (60 episodes/condition). Zero observed duplicates corresponds to a one-sided exact 95% upper bound of 4.87% per condition.

Campaign	WAL	Baseline	Determ. keys
Scripted planner	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000
LLM planner (uniform)	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000
LLM planner (mixed)	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000

TABLE 6
LangGraph v2: duplicate episodes by crash point (60 episodes/condition; crash distribution identical across campaigns by shared seed).

Condition	mid-fan-out	retry-backoff	post-branch	Total
WAL	0/33	0/20	0/7	0/60
Baseline	30/30	18/18	0/12	48/60
Determ. keys	0/32	0/13	0/15	0/60

; zero observed duplicates $\rightarrow$ one-sided exact 95% upper bound 0.20%.

Condition	Duplicate rate	Exactly-once
Full protocol	0.0000	1.0000
No fencing	0.4927	0.5073
No claim log	0.2333	0.7667
Neither	0.8767	0.1233